

Build a production-ready **Customer & Telecaller Ticket Management System**.

The purpose of the application is to allow customers and telecallers to raise support tickets, and allow internal support teams to receive, manage, resolve, escalate, and close those tickets.

1. User Roles

Create role-based access for:

1. Admin

2. Full system access
3. Manage users, departments, categories, priorities, SLAs and system settings
4. View all tickets
5. Assign/reassign tickets
6. View dashboards and reports

7. Support Agent

8. View tickets assigned to them
9. Update ticket status
10. Communicate with customers
11. Add internal notes
12. Request additional information
13. Resolve tickets

14. Team Leader / Manager

15. View team/department tickets
16. Assign and reassign tickets
17. Monitor SLA
18. Escalate tickets
19. View team performance and reports

20. Telecaller

21. Search customers
22. Create tickets on behalf of customers
23. Add call summary and issue details
24. View tickets they have raised
25. Track ticket status
26. Add follow-up notes

27. Follow up with customers

28. **Customer**

29. Create tickets

30. View their own tickets

31. Track ticket status

32. Reply to support

33. Upload attachments

34. Provide additional information

35. Reopen a resolved ticket

36. Give feedback/rating after resolution

2. Core Ticket Workflow

Implement this workflow:

Customer/Telecaller creates ticket → Ticket ID generated automatically → Category and priority selected → Ticket assigned to appropriate department/agent → Agent starts working → Agent communicates with customer → Ticket may move to "Waiting for Customer" → Agent resolves the issue → Customer can confirm resolution → Ticket is closed → Customer provides feedback/rating

If the customer is not satisfied, they should be able to reopen the ticket.

Required statuses:

- New
- Assigned
- In Progress
- Waiting for Customer
- Escalated
- Resolved
- Reopened
- Closed
- Cancelled

3. Ticket Fields

Create a proper ticket model containing at least:

- Ticket ID
- Customer
- Created by
- Source (Customer / Telecaller / Other)
- Category
- Subcategory
- Subject
- Description
- Priority
- Status

- Department
- Assigned agent
- Created date/time
- Updated date/time
- SLA deadline
- SLA status
- Resolution details
- Closed date/time
- Reopen count
- Customer rating
- Customer feedback

Allow attachments such as images, PDFs and documents.

4. Ticket Details Page

Create a professional ticket details screen showing:

- Ticket information
- Customer information
- Current status
- Priority
- Assigned agent
- SLA countdown
- Conversation
- Internal notes
- Attachments
- Activity/audit history
- Resolution details

Keep **internal notes completely separate from customer-visible messages.**

5. Dashboard

Create dashboards appropriate to each role.

Admin/Manager dashboard should show:

- Total tickets
- New tickets
- Open tickets
- In-progress tickets
- Waiting for customer
- Escalated tickets
- Resolved tickets
- Closed tickets
- SLA breached tickets
- Average response time
- Average resolution time
- SLA compliance
- Tickets by category

- Tickets by department
- Tickets by agent

Use clean charts and cards where appropriate.

6. Search and Filtering

Implement powerful ticket search/filtering by:

- Ticket ID
- Customer name
- Phone number
- Email
- Category
- Status
- Priority
- Department
- Assigned agent
- Created date
- SLA status
- Ticket source

7. SLA and Escalation

Create configurable SLA rules based on priority.

Example:

Critical → 1 hour High → 4 hours Medium → 8 hours Low → 24 hours

Do not hard-code these values permanently. Admin should be able to configure them later.

The system should:

- Start SLA when ticket is created
- Display remaining SLA time
- Warn before SLA breach
- Mark SLA as breached
- Support automatic escalation
- Record SLA history

8. Notifications

Create a notification architecture for:

- Ticket creation
- Ticket assignment
- New customer reply
- Agent response
- Status changes
- SLA approaching

- SLA breach
- Ticket escalation
- Ticket resolution
- Ticket closure
- Ticket reopening

Initially implement in-app notifications, but structure the system so email/SMS/WhatsApp can be integrated later.

9. Audit Trail

Every important ticket action must be recorded.

Examples:

- Ticket created
- Ticket assigned
- Agent changed
- Priority changed
- Status changed
- Customer replied
- Agent replied
- Internal note added
- Ticket escalated
- Ticket resolved
- Ticket reopened
- Ticket closed

Store who performed the action and the timestamp.

10. UI/UX

Create a modern, professional SaaS-style interface.

Requirements:

- Responsive design
- Desktop-first but mobile-friendly
- Clean sidebar navigation
- Dashboard cards
- Data tables with pagination
- Search and filters
- Status badges
- Priority indicators
- Confirmation dialogs for important actions
- Loading states
- Empty states
- Error handling
- Form validation

Do not make the UI overly complicated.

11. Architecture

Before writing the application, first define:

- Recommended technology stack
- Frontend architecture
- Backend architecture
- Database schema
- Authentication architecture
- Authorization/RBAC
- API structure
- File storage approach
- Notification architecture
- SLA architecture
- Audit logging architecture

Use a modular architecture so new features can be added later without rewriting the application.

12. Database

Design normalized database tables for:

- Users
- Roles
- Permissions
- Customers
- Departments
- Categories
- Subcategories
- Tickets
- Ticket messages
- Internal notes
- Attachments
- Ticket assignments
- Ticket status history
- SLA rules
- Escalations
- Notifications
- Audit logs
- Customer feedback

Include appropriate primary keys, foreign keys, indexes and timestamps.

13. Security

Implement:

- Secure authentication
- Password hashing
- Role-based authorization
- Input validation

- API authorization
- File upload validation
- Protection against unauthorized ticket access
- Audit logging
- Secure session/token handling

Customers must never be able to access another customer's tickets.

14. Development Approach

Do NOT generate the entire application immediately.

First provide:

1. Recommended tech stack with reasoning
2. High-level system architecture
3. User-role permission matrix
4. Complete database schema
5. Main application modules
6. API endpoint plan
7. Folder/project structure
8. Ticket lifecycle/state diagram
9. Development phases

After presenting this architecture, wait for my approval before implementing the application.

The application should be designed as a scalable production system rather than a simple demo or CRUD prototype.